

# Plan de Reconstrucción

## Módulo IA de Autorizaciones

Guía técnica para implementar desde cero la extracción automática de datos desde autorizaciones de cirugía en el entorno real de desarrollo

### ALCANCE

Este documento define cómo reconstruir la funcionalidad de IA de autorizaciones desde cero en el repositorio principal de OSSUM COR. No es una migración del sandbox: es un plan de reconstrucción limpia que aplica las lecciones aprendidas, evita los errores del prototipo, y sigue las reglas del entorno real de producción.

**Tipo:** Plan de reconstrucción — No es migración

**Origen:** Sandbox validado (Junio 2026)

**Destino:** Repositorio principal OSSUM COR

**Stack:** Next.js 16 · TypeScript · Zod · react-hook-form · z-ai-web-dev-sdk

CHECKLIST EJECUTABLE INCLUIDO

# Contenido

|                                                              |    |
|--------------------------------------------------------------|----|
| 1. Conocimiento que se conserva del sandbox                  | 3  |
| 1.1 Conocimiento de dominio . . . . .                        | 3  |
| 1.2 Prompt VLM validado . . . . .                            | 3  |
| 1.3 Schema Zod validado . . . . .                            | 3  |
| 1.4 Flujo UI validado . . . . .                              | 3  |
| 1.5 Umbrales y constantes . . . . .                          | 4  |
| 2. Que NO debe copiarse del sandbox                          | 4  |
| 3. Modulos reales de OSSUM COR que reciben la funcionalidad  | 5  |
| 4. Archivos reales a crear                                   | 5  |
| 5. Archivos reales a tocar con cuidado                       | 6  |
| 6. Dependencias necesarias                                   | 7  |
| 7. Variables de entorno                                      | 8  |
| 8. Endpoint backend                                          | 9  |
| 8.1 Ruta y metodo . . . . .                                  | 9  |
| 8.2 Request . . . . .                                        | 9  |
| 8.3 Response exitosa (200) . . . . .                         | 9  |
| 8.4 Responses de error . . . . .                             | 9  |
| 8.5 Implementacion critica . . . . .                         | 10 |
| 9. Servicio server-side                                      | 10 |
| 9.1 Interface AIProvider . . . . .                           | 10 |
| 9.2 Factory de providers . . . . .                           | 10 |
| 9.3 Prompt VLM (version mejorada) . . . . .                  | 11 |
| 10. Schema Zod                                               | 12 |
| 10.1 autorizacion.ts — Schema de extraccion IA . . . . .     | 12 |
| 10.2 cirugia.ts — Schema del formulario de cirugia . . . . . | 13 |

|                                                    |    |
|----------------------------------------------------|----|
| 11. Mapeo del JSON al wizard real                  | 14 |
| 11.1 Mapa declarativo de campos . . . . .          | 14 |
| 12. Validaciones minimas                           | 16 |
| 13. Errores a evitar                               | 17 |
| 14. Pasos en orden                                 | 17 |
| 15. Como probar en local                           | 19 |
| 15.1 Prueba del endpoint (curl) . . . . .          | 19 |
| 15.2 Prueba del schema Zod (Jest/Vitest) . . . . . | 20 |
| 15.3 Prueba visual del formulario . . . . .        | 20 |
| 15.4 Prueba de errores . . . . .                   | 20 |
| 16. Como documentar en worklog/ADR                 | 21 |
| 16.1 ADR (Architecture Decision Records) . . . . . | 21 |
| 16.2 Worklog . . . . .                             | 21 |
| 16.3 README del modulo . . . . .                   | 21 |
| Checklist ejecutable para Codex/OpenCode           | 22 |

# 1. Conocimiento que se conserva del sandbox

El sandbox fue un entorno de validacion donde se demostro que la extraccion de datos desde autorizaciones de cirugia mediante un Vision-Language Model es funcional, viable y rapida. No se copia codigo, pero se conserva el conocimiento adquirido. A continuacion se detalla exactamente que saber es transferible y por que.

## 1.1 Conocimiento de dominio

El sandbox confirmo que las autorizaciones de ART/obra social argentinas tienen un conjunto predecible de campos: paciente, DNI, medico, institucion, obra social, numero de autorizacion, numero de siniestro, numero de poliza, fechas (autorizacion y cirugia), listado de material autorizado con codigos, descripciones, cantidades y precios de referencia, y observaciones. Este dominio semi-estructurado es ideal para extraccion por VLM porque el modelo "sabe" donde buscar y que formato esperar, incluso con documentos nunca vistos. Este conocimiento de dominio es 100% transferible y es la base del prompt que se disene para produccion.

## 1.2 Prompt VLM validado

El prompt utilizado en sandbox fue funcional: el VLM extrajo correctamente datos de autorizaciones reales. La estructura del prompt (instruccion en espanol + schema JSON embebido como template + campo de confianza + campo de validacion de tipo de documento) es el patron que debe conservarse. El texto exacto del prompt se puede mejorar, pero la estructura esta validada. Ver Seccion 9 para el prompt completo y las mejoras sugeridas.

## 1.3 Schema Zod validado

Los schemas Zod (MaterialAutorizadoItemSchema, AutorizacionExtractedSchema, AutorizacionAIResponseSchema) fueron probados con respuestas reales del VLM y funcionan correctamente. La estructura de campos, los defaults y los tipos son adecuados para el dominio. Los schemas se deben recrear (no copiar) pero pueden seguir la misma estructura con mejoras menores (ver Seccion 10).

## 1.4 Flujo UI validado

El flujo de usuario fue validado: upload de archivo, procesamiento con IA, visualizacion de resultados, aplicacion al formulario, validacion humana, guardado. Este flujo es correcto y debe replicarse. Lo que cambia es la implementacion: el sandbox uso un componente monolitico de 964 lineas en page.tsx con useState crudo; el entorno real debe usar componentes desacoplados, react-hook-form y zustand para estado.

## 1.5 Umbrales y constantes

Varios umbrales fueron determinados empíricamente en sandbox y son directamente transferibles: tamaño máximo de archivo 20 MB, tipos permitidos (PDF, JPG, PNG, WebP, BMP), umbral de confianza para fallback (0.3), campo confidence con default 0.7 si el VLM no lo devuelve, campo looks\_like\_authorization con default true si falta. Estos valores surgieron de prueba y error y no necesitan rediseñarse.

## 2. Que NO debe copiarse del sandbox

**ADVERTENCIA:** El sandbox fue un prototipo rápido. Copiar su código al entorno real introduciría deuda técnica, bugs conocidos y patrones incorrectos. A continuación se lista todo lo que debe descartarse y por qué.

| Componente del sandbox           | Razon para no copiar                                                                                              | Reemplazo en entorno real                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| page.tsx monolitico (964 lineas) | Acoplamiento total. Logica de UI, estado, IA, form y API calls en un solo componente. Inmantenible, no testeable. | Componentes desacoplados: SurgeryForm, AiUploadZone, AiResultsPanel, MaterialTable.     |
| useState crudo para formulario   | Sin validacion, sin dirty state tracking, sin manejo de errores por campo. Bug de normalizeDate() no detectado.   | react-hook-form con resolver Zod. Validacion por campo, dirty tracking, error messages. |
| Store in-memory para cirugias    | Se pierde al reiniciar. No escala. No consulta. No filtra.                                                        | Prisma model Surgery (cuando se decida) o almacenamiento real.                          |
| Regex fallback sin OCR           | Funcionalmente inutil: opera sobre placeholder string. Daba falsa sensacion de seguridad.                         | No implementar fallback hasta tener capa OCR real, o eliminarlo.                        |
| MOCK_EXTRACTED hardcodeado       | Datos de prueba contaminan la logica de produccion. El modo mock es util pero no asi.                             | Provider mock como estrategia pattern, separado del extractor real.                     |
| Progreso simulado                | Barra que incrementa aleatoriamente. No refleja progreso real.                                                    | Usar estados concretos: uploading, processing, complete, error.                         |
| Tipos duplicados                 | SurgeryFormData en page.tsx repite campos de AutorizacionExtracted en Zod. Dos fuentes de verdad.                 | Un solo schema Zod como fuente de verdad. Tipos derivados con z.infer.                  |
| API route /lab/ai/autorizacion   | Namespace "lab" es de sandbox. No corresponde a la estructura del ERP real.                                       | Ruta propia bajo /api/cirugias/ o modulo apropiado del ERP.                             |
| ZAI.create() sin config          | SDK inicializado sin opciones de timeout, retries, ni provider selection.                                         | Factory con config centralizada, timeout, y provider configurable.                      |

Tabla 1: Componentes del sandbox que NO se copian y sus reemplazos

### 3. Modulos reales de OSSUM COR que reciben la funcionalidad

En el entorno real de OSSUM COR, la funcionalidad de IA de autorizaciones se integra en modulos existentes o nuevos. A continuacion se mapea cada pieza funcional al modulo que la debe contener. Este mapeo asume una estructura Next.js App Router con separacion por dominio funcional.

| Modulo                    | Ruta sugerida                  | Responsabilidad                                                                       |
|---------------------------|--------------------------------|---------------------------------------------------------------------------------------|
| Cirugias (nuevo)          | /src/app/(dashboard)/cirugias/ | Rutas de cirugias: listado, nueva, detalle. El wizard de Nueva Cirugia vive aqui.     |
| AI Service (nuevo)        | /src/lib/services/ai/          | Logica de extraccion: VLM, providers, factory, prompt templates.                      |
| Validators (existente)    | /src/lib/validators/           | Schemas Zod para autorizacion y cirugia. Tipos derivados.                             |
| Components (nuevo subdir) | /src/components/cirugias/      | Componentes UI especificos: SurgeryForm, AiUploadZone, AiResultsPanel, MaterialTable. |
| API Cirugias (nuevo)      | /src/app/api/cirugias/         | Endpoints REST: POST /cirugias, GET /cirugias, POST /cirugias/ai-extract.             |
| Hooks (nuevo)             | /src/hooks/                    | useSurgeryForm, useAiExtraction, useSurgeryMutations.                                 |
| Stores (nuevo)            | /src/lib/stores/               | Zustand store para estado global de cirugia si se necesita.                           |

Tabla 2: Modulos del ERP y su responsabilidad

### 4. Archivos reales a crear

A continuacion se lista cada archivo que debe crearse desde cero en el entorno real, con su ruta, funcion y prioridad de creacion. Los archivos estan ordenados por dependencia: los primeros no dependen de nadie, los ultimos dependen de los anteriores.

| Pa so | Archivo                            | Funcion                                                                                                              | Prioridad |
|-------|------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------|
| 1     | src/lib/validators/autorizacion.ts | Schema Zod: AutorizacionExtracted, AutorizacionAIResponse, MaterialAutorizadolItem. Tipos derivados. Normalizadores. | Alta      |
| 2     | src/lib/validators/cirugia.ts      | Schema Zod: SurgeryFormSchema (campos del wizard). Tipos derivados para el formulario.                               | Alta      |

|    |                                                |                                                                                         |       |
|----|------------------------------------------------|-----------------------------------------------------------------------------------------|-------|
| 3  | src/lib/services/ai/providers/types.ts         | Interface AIProvider con metodo extract(). Define el contrato para cualquier proveedor. | Alta  |
| 4  | src/lib/services/ai/providers/z-ai-provider.ts | Implementacion de AIProvider con z-ai-web-dev-sdk. Prompt, parsing, manejo de errores.  | Alta  |
| 5  | src/lib/services/ai/providers/mock-provider.ts | Implementacion de AIProvider para testing sin consumo de IA.                            | Media |
| 6  | src/lib/services/ai/autorizacion-extractor.ts  | Funcion extractAutorizacion() que usa el factory de providers. Logica de fallback.      | Alta  |
| 7  | src/lib/services/ai/config.ts                  | Configuracion centralizada: provider activo, timeout, retries, constantes.              | Alta  |
| 8  | src/app/api/cirugias/ai-extract/route.ts       | Endpoint POST que recibe archivo y mode, invoca extractor, retorna JSON validado.       | Alta  |
| 9  | src/hooks/useAiExtraction.ts                   | Hook que encapsula el fetch al endpoint, maneja estados (idle/loading/success/error).   | Media |
| 10 | src/components/cirugias/AiUploadZone.tsx       | Componente de upload con drag-and-drop, preview, y feedback de procesamiento.           | Media |
| 11 | src/components/cirugias/AiResultsPanel.tsx     | Componente que muestra resultados IA: confianza, warnings, preview, boton "Aplicar".    | Media |
| 12 | src/components/cirugias/SurgeryForm.tsx        | Formulario de cirugia con react-hook-form + Zod resolver. Todos los campos.             | Alta  |
| 13 | src/components/cirugias/MaterialTable.tsx      | Tabla dinamica de material autorizado (agregar/quitar items, campos editables).         | Media |
| 14 | src/app/(dashboard)/cirugias/nueva/page.tsx    | Pagina Nueva Cirugia que compone los componentes anteriores.                            | Alta  |
| 15 | src/app/api/cirugias/route.ts                  | Endpoint POST/GET para crear/listar cirugias. Inicialmente in-memory, luego DB.         | Media |

Tabla 3: Archivos a crear, ordenados por dependencia

## 5. Archivos reales a tocar con cuidado

**ADVERTENCIA:** Estos archivos ya existen en el entorno real. Modificarlos incorrectamente puede romper funcionalidad existente. Se listan solo los cambios necesarios con el impacto minimo.

| Archivo | Cambio requerido | Riesgo | Mitigacion |
|---------|------------------|--------|------------|
|---------|------------------|--------|------------|

|                                |                                                                                              |         |                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------|
| .env / .env.local              | Agregar AI_PROVIDER, AI_TIMEOUT, variables de provider (solo si aplica).                     | Bajo    | Solo agregar, nunca eliminar vars existentes. Documentar cada nueva variable. |
| src/app/layout.tsx             | Posiblemente agregar providers (ReactQuery, Theme) si el ERP real no los tiene.              | Medio   | Verificar que providers existan primero. Agregar solo los necesarios.         |
| src/lib/db.ts                  | Sin cambios inmediatos. Cuando se modele Surgery en Prisma, se usara el singleton existente. | Ninguno | No tocar hasta decision explicita de modelar Surgery.                         |
| prisma/schema.prisma           | Sin cambios inmediatos. Solo agregar modelo Surgery cuando se decida persistir.              | Alto    | No agregar modelos hasta decision explicita. No tocar User/Post.              |
| package.json                   | Verificar que z-ai-web-dev-sdk, zod, react-hook-form, @hookform/resolvers esten instalados.  | Bajo    | Solo agregar dependencias faltantes. No eliminar ninguna.                     |
| next.config.ts                 | Posiblemente aumentar bodySizeLimit para uploads grandes si Next.js 16 lo requiere.          | Bajo    | Verificar limite default primero. Solo agregar si los uploads fallan.         |
| src/app/(dashboard)/layout.tsx | Si existe, agregar link a /cirugias/nueva en la navegacion lateral.                          | Medio   | Solo agregar un link, no reestructurar navegacion.                            |

Tabla 4: Archivos existentes a tocar con cuidado

## 6. Dependencias necesarias

La siguiente tabla lista las dependencias necesarias para el modulo, clasificadas por si ya estan instaladas en el proyecto real o si hay que agregarlas. Se prioriza reutilizar lo que ya existe antes de instalar paquetes nuevos.

| Paquete             | Ya instalado? | Uso en el modulo                                              | Accion                                            |
|---------------------|---------------|---------------------------------------------------------------|---------------------------------------------------|
| z-ai-web-dev-sdk    | Si            | Proveedor VLM principal. createVision() para extraccion.      | Verificar version >= 0.0.17.                      |
| zod                 | Si            | Validacion de schemas en backend y frontend. Tipos derivados. | Verificar version >= 4.0.                         |
| react-hook-form     | Si            | Manejo del formulario de cirugia con validacion por campo.    | Verificar que se use con @hookform/resolvers/zod. |
| @hookform/resolvers | Si            | Integracion Zod + react-hook-form.                            | Verificar version compatible con Zod 4.           |

|                       |    |                                                                           |                                                               |
|-----------------------|----|---------------------------------------------------------------------------|---------------------------------------------------------------|
| zustand               | Si | Estado global (si se necesita compartir estado de cirugia entre paginas). | Evaluar si es necesario o si alcanza con estado local de RHF. |
| @tanstack/react-query | Si | Fetch con cache, retry automatico, estados de loading/error.              | Usar para el hook useAiExtraction en lugar de fetch crudo.    |
| sharp                 | Si | Conversion PDF a imagen (futuro). No se usa en la implementacion inicial. | No instalar nada nuevo. Dejar instalado para uso futuro.      |
| lucide-react          | Si | Iconos en componentes UI.                                                 | Sin cambios.                                                  |
| date-fns              | Si | Parseo y formateo de fechas (normalizacion DD/MM/YYYY a YYYY-MM-DD).      | Usar en lugar de normalizeDate() custom.                      |

Tabla 5: Dependencias del modulo

**Conclusion:** Todas las dependencias ya estan instaladas en el proyecto. No se requiere instalar ningun paquete nuevo. El trabajo es configurar y usar lo que ya existe, particularmente react-hook-form (que esta instalado pero no se uso en sandbox) y @tanstack/react-query (idem).

## 7. Variables de entorno

Las variables de entorno deben definirse antes de la implementacion. El principio es: ningun valor hardcoded, toda configuracion via variables de entorno con defaults seguros. A continuacion se listan las variables necesarias, sus valores por defecto y si son obligatorias.

| Variable                | Default | Obligatoria                      | Descripcion                                                                                   |
|-------------------------|---------|----------------------------------|-----------------------------------------------------------------------------------------------|
| AI_PROVIDER             | z-ai    | No                               | Proveedor de IA activo. Valores: "z-ai"   "mock". Futuro: "openai"   "anthropic".             |
| AI_TIMEOUT_MS           | 30000   | No                               | Timeout en milisegundos para llamadas al proveedor de IA.                                     |
| AI_MAX_FILE_SIZE_MB     | 20      | No                               | Tamano maximo de archivo en megabytes.                                                        |
| AI_CONFIDENCE_THRESHOLD | 0.3     | No                               | Umbral de confianza bajo el cual se activa fallback o se advierte al usuario.                 |
| ZAI_API_KEY             | (auto)  | No en sandbox / Si en produccion | API key para z-ai-web-dev-sdk. En sandbox se autodescubre; en produccion puede ser necesaria. |
| OPENAI_API_KEY          | -       | No                               | API key para OpenAI. Solo si AI_PROVIDER=openai.                                              |
| ANTHROPIC_API_KEY       | -       | No                               | API key para Anthropic. Solo si AI_PROVIDER=anthropic.                                        |

Tabla 6: Variables de entorno del modulo

**Regla critica:** Nunca hardcodear API keys en el codigo fuente. Todas las claves deben venir de variables de entorno. En caso de usar z-ai-web-dev-sdk en entorno Z.ai, la autenticacion es automatica y no se necesita API key explicita. Para otros proveedores, la key correspondiente es obligatoria y debe configurarse antes del deploy.

## 8. Endpoint backend

Se necesita un unico endpoint para la extraccion de datos con IA. El endpoint recibe un archivo y un modo, invoca el servicio de IA y retorna JSON validado. A continuacion se define la interfaz completa del endpoint.

### 8.1 Ruta y metodo

```
POST /api/cirugias/ai-extract
```

### 8.2 Request

Content-Type: multipart/form-data

| Campo | Tipo          | Requerido                       | Descripcion                                                            |
|-------|---------------|---------------------------------|------------------------------------------------------------------------|
| file  | File (binary) | Si                              | Archivo de autorizacion. Formatos: PDF, JPG, PNG, WebP, BMP. Max 20MB. |
| mode  | string        | No (default: segun AI_PROVIDER) | Modo de procesamiento: "vlm" o "mock". Si se omite, usa AI_PROVIDER.   |

Tabla 7: Campos del request

### 8.3 Response exitosa (200)

Content-Type: application/json. Retorna AutorizacionAIResponse validado por Zod.

El schema de respuesta es identico al validado en sandbox: provider (enum), confidence (0-1), looks\_like\_authorization (boolean), warnings (string[]), extracted (AutorizacionExtracted con 12 campos + material\_autorizado array), raw\_text\_preview (string). Ver Seccion 10 para el schema Zod completo.

### 8.4 Responses de error

| Status | Condicion                | Body                                           |
|--------|--------------------------|------------------------------------------------|
| 400    | Archivo no proporcionado | { "error": "No se proporciono archivo" }       |
| 400    | Tipo no soportado        | { "error": "Tipo de archivo no soportado..." } |

|     |                       |                                                    |
|-----|-----------------------|----------------------------------------------------|
| 400 | Archivo excede limite | { "error": "El archivo supera el limite de 20MB" } |
| 408 | Timeout de IA         | { "error": "Timeout: la IA no respondio en 30s" }  |
| 500 | Error interno         | { "error": "Error al procesar: " }                 |

Tabla 8: Respuestas de error del endpoint

## 8.5 Implementacion critica

- **Timeout:** El endpoint DEBE implementar un timeout para la llamada al proveedor de IA. El sandbox no lo tenia, causando que requests colgados nunca se resuelvan. Usar AbortController o Promise.race con un setTimeout.
- **Validacion antes de procesar:** Verificar tipo MIME + extension. Verificar tamano. Rechazar temprano antes de consumir recursos del servidor.
- **No logear datos sensibles:** El archivo puede contener datos de salud (HIPAA/PDP). No logear el base64 ni el contenido extraido. Solo logear metadata (tamano, tipo, provider, confidence, duracion).
- **Idempotencia:** El endpoint es stateless (no guarda nada). Es seguro reintentar.

## 9. Servicio server-side

El servicio server-side es el core del modulo. A diferencia del sandbox donde toda la logica estaba en un solo archivo, el entorno real debe usar un patron Strategy para soportar multiples proveedores de IA sin acoplamiento.

### 9.1 Interface AIProvider

```
interface AIProviderConfig {
  timeout: number; // ms
  maxRetries: number;
}

interface AIProvider {
  readonly name: string;
  extract(base64Image: string, mimeType: string): Promise<AutorizacionAIResponse>;
}
```

### 9.2 Factory de providers

```
// src/lib/services/ai/config.ts
export const aiConfig = {
```

```

provider: (process.env.AI_PROVIDER as string) || 'z-ai',
timeout: parseInt(process.env.AI_TIMEOUT_MS || '30000'),
maxFileSize: parseInt(process.env.AI_MAX_FILE_SIZE_MB || '20') * 1024 * 1024,
confidenceThreshold: parseFloat(process.env.AI_CONFIDENCE_THRESHOLD || '0.3'),
};

// src/lib/services/ai/autorizacion-extractor.ts
import { zAiProvider } from './providers/z-ai-provider';
import { mockProvider } from './providers/mock-provider';

const PROVIDERS: Record<string, AIPProvider> = {
  'z-ai': zAiProvider,
  'mock': mockProvider,
  // Futuro: 'openai': openAiProvider,
  // Futuro: 'anthropic': anthropicProvider,
};

export async function extractAutorizacion(
  buffer: Buffer, mimeType: string, mode?: string
): Promise<AutorizacionAIResponse> {
  const providerName = mode === 'mock' ? 'mock' : aiConfig.provider;
  const provider = PROVIDERS[providerName];
  if (!provider) throw new Error(`AI provider "${providerName}" not found`);

  const base64 = buffer.toString('base64');
  return provider.extract(base64, mimeType);
}

```

## 9.3 Prompt VLM (version mejorada)

El prompt del sandbox fue funcional pero puede mejorarse. A continuacion se presenta la version mejorada que agrega: (a) instrucciones de formato de fecha, (b) manejo de tablas de material, (c) campo de tipo de documento para futura clasificacion.

```

Extrae los datos de esta autorizacion de cirugia / ART / obra social.

REGLAS:
- Responde SOLO con JSON valido, sin markdown ni explicaciones.
- Las fechas deben estar en formato YYYY-MM-DD.
- Si un campo no aparece en el documento, dejalo vacio ("").
- material_autorizado: extrae CADA item de la tabla de materiales, incluyendo
codigo, descripcion, cantidad y precio.
- Si el documento NO es una autorizacion de cirugia, pon
looks_like_authorization en false.

```

- confidence: tu nivel de confianza en la extraccion (0.0 a 1.0).

JSON de respuesta:

```
{
  "paciente": "",
  "dni": "",
  "medico": "",
  "institucion": "",
  "obra_social": "",
  "numero_autorizacion": "",
  "numero_siniestro": "",
  "numero_poliza": "",
  "fecha_autorizacion": "",
  "fecha_cirugia": "",
  "material_autorizado": [{"codigo": "", "descripcion": "", "cantidad": "", "precio_referencia": ""}],
  "observaciones": "",
  "looks_like_authorization": true/false,
  "confidence": 0.0-1.0
}
```

**Cambios vs. sandbox:** Se agregaron REGLAS explicitas (fechas en YYYY-MM-DD, campo vacio si no existe, cada item de tabla), se elimino el voseo argentina ("Extrae" en lugar de "Extrae" con acento) para mayor compatibilidad con modelos, y se anadio la instruccion de extraer CADA item de la tabla de materiales para mejorar la extraccion en documentos con tablas complejas.

## 10. Schema Zod

Los schemas Zod son la fuente de verdad para los tipos de datos. Tanto el backend como el frontend derivan sus tipos de estos schemas. A continuacion se definen los schemas necesarios con las mejoras respecto al sandbox.

### 10.1 autorizacion.ts — Schema de extraccion IA

```
// src/lib/validators/autorizacion.ts
import { z } from 'zod';

export const MaterialAutorizadoItemSchema = z.object({
  codigo: z.string().default(''),
  descripcion: z.string().default(''),
  cantidad: z.string().default(''), // string para permitir "2-3", "4+"
});
```

```

precio_referencia: z.string().default(''), // string para "$85.000"
});

export const AutorizacionExtractedSchema = z.object({
  paciente: z.string().default(''),
  dni: z.string().default(''),
  medico: z.string().default(''),
  institucion: z.string().default(''),
  obra_social: z.string().default(''),
  numero_autorizacion: z.string().default(''),
  numero_siniestro: z.string().default(''),
  numero_poliza: z.string().default(''),
  fecha_autorizacion: z.string().default(''), // YYYY-MM-DD
  fecha_cirugia: z.string().default(''), // YYYY-MM-DD
  material_autorizado: z.array(MaterialAutorizadoItemSchema).default([]),
  observaciones: z.string().default(''),
});

export const AutorizacionAIResponseSchema = z.object({
  provider: z.enum(['z-ai', 'mock', 'fallback-regex']),
  confidence: z.number().min(0).max(1).default(0),
  looks_like_authorization: z.boolean().default(false),
  warnings: z.array(z.string()).default([]),
  extracted: AutorizacionExtractedSchema,
  raw_text_preview: z.string().default(''),
});

// Tipos derivados
export type MaterialAutorizadoItem = z.infer<typeof
MaterialAutorizadoItemSchema>;

export type AutorizacionExtracted = z.infer<typeof AutorizacionExtractedSchema>;
export type AutorizacionAIResponse = z.infer<typeof
AutorizacionAIResponseSchema>;

```

## 10.2 cirugia.ts — Schema del formulario de cirugia

```

// src/lib/validators/cirugia.ts
import { z } from 'zod';
import { MaterialAutorizadoItemSchema } from './autorizacion';

export const SurgeryFormSchema = z.object({
  paciente: z.string().min(1, 'Nombre del paciente es requerido'),
  dni: z.string().min(7, 'DNI invalido').max(8, 'DNI invalido'),

```

```

medico: z.string().min(1, 'Medico es requerido'),
institucion: z.string().min(1, 'Institucion es requerida'),
obra_social: z.string().min(1, 'Obra social es requerida'),
numero_autorizacion: z.string().min(1, 'Numero de autorizacion requerido'),
numero_siniestro: z.string().default(''),
numero_poliza: z.string().default(''),
fecha_autorizacion: z.string().min(1, 'Fecha requerida'),
fecha_cirugia: z.string().min(1, 'Fecha requerida'),
material_autorizado: z.array(MaterialAutorizadoItemSchema).min(1, 'Al menos un item'),
observaciones: z.string().default(''),
});

export type SurgeryFormData = z.infer<typeof SurgeryFormSchema>;

```

**Cambios vs. sandbox:** Se unifico la fuente de verdad (un solo schema para campos de autorizacion, no dos tipos duplicados). Se agregaron validaciones min() a campos obligatorios. Se elimino SurgeryFormData como interfaz separada; ahora se deriva de Zod con z.infer. El enum de provider cambio de "z-ai-vlm" a "z-ai" para ser generico.

## 11. Mapeo del JSON al wizard real

El mapeo entre el JSON extraido por la IA y el formulario de cirugia debe ser explicito, bidireccional y testeable. En el sandbox, el mapeo estaba hardcodedo en applyAiToForm() con asignaciones individuales. En el entorno real, se debe usar un mapa declarativo.

### 11.1 Mapa declarativo de campos

```

// src/lib/validators/autorizacion.ts (agregar al final)
import { parse, format } from 'date-fns';

export const AUTORIZACION_TO_SURGERY_MAP = {
  paciente: 'paciente',
  dni: 'dni',
  medico: 'medico',
  institucion: 'institucion',
  obra_social: 'obra_social',
  numero_autorizacion: 'numero_autorizacion',
  numero_siniestro: 'numero_siniestro',
  numero_poliza: 'numero_poliza',
  fecha_autorizacion: 'fecha_autorizacion',

```

```

fecha_cirugia: 'fecha_cirugia',
material_autorizado: 'material_autorizado',
observaciones: 'observaciones',
} as const;

// Normalizadores por campo
export const FIELD_NORMALIZERS: Record<string, (v: string) => string> = {
dni: (v) => v.replace(/[.\-]/g, '').replace(/[^\d]/g, ''),
fecha_autorizacion: (v) => normalizeDateToISO(v),
fecha_cirugia: (v) => normalizeDateToISO(v),
};

function normalizeDateToISO(value: string): string {
if (!value) return '';
try {
// Intentar DD/MM/YYYY
const d = parse(value, 'dd/MM/yyyy', new Date());
if (isValid(d)) return format(d, 'yyyy-MM-dd');
} catch {}
try {
// Intentar YYYY-MM-DD (ya normalizado)
const d = parse(value, 'yyyy-MM-dd', new Date());
if (isValid(d)) return value;
} catch {}
return value; // Devolver original si no se puede normalizar
}

export function mapAutorizacionToSurgery(
extracted: AutorizacionExtracted
): Partial<SurgeryFormData> {
const result: Record<string, unknown> = {};
for (const [srcKey, dstKey] of Object.entries(AUTORIZACION_TO_SURGERY_MAP)) {
const raw = extracted[srcKey as keyof AutorizacionExtracted];
const normalizer = FIELD_NORMALIZERS[srcKey];
if (normalizer && typeof raw === 'string') {
result[dstKey] = normalizer(raw);
} else {
result[dstKey] = raw;
}
}
}

// Garantizar al menos 1 item en material_autorizado

```

```

if (Array.isArray(result.material_autorizado) &&
result.material_autorizado.length === 0) {
  result.material_autorizado = [{ codigo: '', descripcion: '', cantidad: '',
precio_referencia: '' }];
}
return result as Partial<SurgeryFormData>;
}

```

**Cambios vs. sandbox:** El mapeo es declarativo (un objeto en vez de 12 asignaciones), los normalizadores se aplican por campo (corrigiendo el bug de `normalizeDate()`), y se usa `date-fns` para parseo robusto de fechas. La funcion `mapAutorizacionToSurgery()` es testeable unitariamente con datos de entrada y salida esperados.

## 12. Validaciones minimas

Las validaciones se aplican en tres capas: backend (endpoint), servicio (extractor), y frontend (formulario). Cada capa tiene su responsabilidad y no debe duplicar la de la otra.

| Capa                  | Validacion                      | Razon                                                                       |
|-----------------------|---------------------------------|-----------------------------------------------------------------------------|
| Backend (endpoint)    | Tipo MIME permitido + extension | Rechazar archivos no procesables antes de consumir recursos.                |
| Backend (endpoint)    | Tamano maximo (20MB)            | Prevenir uploads que agoten memoria del servidor.                           |
| Backend (endpoint)    | Timeout (30s)                   | Evitar que un request colgado consuma un worker indefinitely.               |
| Servicio (extractor)  | Zod parse de respuesta IA       | Garantizar que el JSON cumple el schema antes de enviar al frontend.        |
| Servicio (extractor)  | Confidence >= threshold         | Si confidence < 0.3, agregar warning. No bloquear, solo informar.           |
| Frontend (formulario) | react-hook-form + Zod resolver  | Validacion por campo en tiempo real. Campos requeridos, DNI valido, fechas. |
| Frontend (formulario) | Al menos 1 item de material     | No permitir guardar cirugia sin material autorizado.                        |
| Frontend (formulario) | Confirmacion humana             | Boton "Aplicar al formulario" como paso explicito. No autoguardar IA.       |

Tabla 9: Validaciones por capa

## 13. Errores a evitar

**ADVERTENCIA:** Estos son los errores concretos que se cometieron en el sandbox. Cada uno debe evitarse en la reconstruccion real. Se documenta el error, la causa raiz y la prevencion.

| Error del sandbox                                          | Causa raiz                                                                         | Prevencion en entorno real                                                           |
|------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| normalizeDate() existe pero no se usa                      | Mapeo con asignaciones individuales donde es facil olvidar una linea.              | Mapa declarativo con normalizadores por campo. Tests unitarios del mapeo.            |
| Regex fallback no funciona                                 | Se implemento sin capa OCR. El texto de entrada era un placeholder.                | No implementar fallback hasta tener OCR real, o eliminar el fallback.                |
| Tipos duplicados (SurgeryFormData + AutorizacionExtracted) | Dos interfaces independientes para los mismos campos. Dos fuentes de verdad.       | Un solo schema Zod. Tipos derivados con z.infer.                                     |
| Componente monolitico de 964 lineas                        | Desarrollo rapido sin refactor. La presion del sandbox justificaba.                | Componentes desacoplados desde el inicio. Max ~150 lineas por componente.            |
| Progreso simulado                                          | Barra que no refleja estado real. Engana al usuario.                               | Estados concretos: idle / uploading / processing / success / error.                  |
| Store in-memory                                            | Solucion rapida para demo. No pensado para persistir.                              | No guardar nada hasta tener modelo DB. O usar localStorage como intermedio.          |
| PDF enviado como base64 sin conversion                     | Se paso el buffer crudo al VLM. Funciona para imagenes, falla para PDFs complejos. | Detectar mimeType y rechazar PDFs o convertir con sharp/libreoffice antes de enviar. |
| Sin timeout en llamada IA                                  | El SDK no tiene timeout por defecto. Un request colgado cuelga al usuario.         | AbortController con setTimeout. Promise.race contra timeout.                         |
| API keys no configurables                                  | z-ai-web-dev-sdk se autentica automaticamente en sandbox. No en produccion.        | Factory de providers con config desde env vars. Nunca hardcodear.                    |

Tabla 10: Errores del sandbox y prevenciones

## 14. Pasos en orden

A continuacion se detallan los pasos de implementacion en orden exacto, con dependencias explícitas. Cada paso produce un artefacto verificable. No avanzar al paso siguiente sin verificar el anterior.

### PASO 1: Configuracion de entorno

Crear .env.local con AI\_PROVIDER=mock, AI\_TIMEOUT\_MS=30000, AI\_MAX\_FILE\_SIZE\_MB=20, AI\_CONFIDENCE\_THRESHOLD=0.3. Verificar que la app compila y corre con las nuevas vars. No cambiar .env existente.

## **PASO 2: Schemas Zod**

Crear `src/lib/validators/autorizacion.ts` con todos los schemas y tipos derivados. Crear `src/lib/validators/cirugia.ts` con `SurgeryFormSchema`. Escribir 1 test unitario que valide que el schema parsea un JSON de ejemplo correctamente y que rechaza un JSON invalido.

## **PASO 3: Interface AIProvider + MockProvider**

Crear `src/lib/services/ai/providers/types.ts` con la interface. Crear `src/lib/services/ai/providers/mock-provider.ts` que implemente la interface con datos de prueba. Verificar que `mockProvider.extract()` retorna un `AutorizacionAIResponse` valido.

## **PASO 4: Config y Factory**

Crear `src/lib/services/ai/config.ts` con la configuracion centralizada. Crear `src/lib/services/ai/autorizacion-extractor.ts` con la funcion `extractAutorizacion()` que usa el factory de providers. Verificar que con `AI_PROVIDER=mock` funciona end-to-end.

## **PASO 5: Endpoint API**

Crear `src/app/api/cirugias/ai-extract/route.ts`. Implementar POST con: validacion de archivo, timeout con `AbortController`, llamada a `extractAutorizacion()`, validacion Zod de respuesta, retorno de JSON. Probar con curl: `curl -F file=@test.jpg -F mode=mock localhost:3000/api/cirugias/ai-extract`

## **PASO 6: ZAI Provider**

Crear `src/lib/services/ai/providers/z-ai-provider.ts`. Implementar `extract()` usando `z-ai-web-dev-sdk` `createVision()` con el prompt mejorado. Agregar al factory. Probar con `AI_PROVIDER=z-ai` y un archivo de imagen real.

## **PASO 7: Hook useAiExtraction**

Crear `src/hooks/useAiExtraction.ts`. Encapsular fetch al endpoint, manejar estados (idle/loading/success/error), y retornar { `extract`, `result`, `error`, `isProcessing` }. Usar `@tanstack/react-query` `useMutation` si esta disponible.

## **PASO 8: Componentes UI**

Crear `src/components/cirugias/AiUploadZone.tsx` (drag-and-drop + file picker). Crear `src/components/cirugias/AiResultsPanel.tsx` (confianza, warnings, preview, boton Aplicar). Crear `src/components/cirugias/MaterialTable.tsx` (tabla dinamica de items). Cada componente debe ser testeable de forma aislada.

## **PASO 9: SurgeryForm con react-hook-form**

Crear `src/components/cirugias/SurgeryForm.tsx` usando `react-hook-form` con `zodResolver`. Todos los campos del schema. Integrar `MaterialTable` como campo del formulario. El formulario debe recibir `initialValues` opcional para precarga desde IA.

### PASO 10: Pagina Nueva Cirugia

Crear `src/app/(dashboard)/cirugias/nueva/page.tsx` que componga `AiUploadZone`, `AiResultsPanel` y `SurgeryForm`. Implementar el flujo: upload, ver resultados, aplicar al formulario, editar, guardar. Usar `mapAutorizacionToSurgery()` para el mapeo.

### PASO 11: Endpoint de guardado

Crear `src/app/api/cirugias/route.ts` con POST para guardar cirugia. Inicialmente usar store in-memory o `localStorage` como intermedio. No tocar Prisma hasta decision explicita. Integrar con el boton "Guardar" del formulario.

### PASO 12: Verificacion end-to-end

Probar el flujo completo: subir imagen, extraer con IA, aplicar al formulario, editar, guardar. Probar con modo mock y con modo z-ai. Probar con archivo invalido, archivo grande, timeout. Verificar que el formulario no se rompe con datos de IA incompletos.

### PASO 13: Documentacion y ADR

Escribir ADR-001: Decision de usar VLM para extraccion de autorizaciones. Escribir ADR-002: Strategy Pattern para providers de IA. Actualizar worklog con cada paso completado. Documentar variables de entorno en README del modulo.

## 15. Como probar en local

A continuacion se detalla como verificar cada componente del modulo en el entorno local de desarrollo, sin depender de servicios externos ni de datos de produccion.

### 15.1 Prueba del endpoint (curl)

```
# Modo mock (sin IA real)
curl -X POST http://localhost:3000/api/cirugias/ai-extract \
-F "file=@test-authorization.jpg" \
-F "mode=mock"

# Modo VLM (con IA real, requiere AI_PROVIDER=z-ai)
curl -X POST http://localhost:3000/api/cirugias/ai-extract \
-F "file=@test-authorization.jpg" \
-F "mode=vlm"

# Archivo invalido (debe retornar 400)
curl -X POST http://localhost:3000/api/cirugias/ai-extract \
-F "file=@test.txt"
```

```
# Sin archivo (debe retornar 400)
curl -X POST http://localhost:3000/api/cirugias/ai-extract
```

## 15.2 Prueba del schema Zod (Jest/Vitest)

```
import { AutorizacionAIResponseSchema, mapAutorizacionToSurgery } from '
@/lib/validators/autorizacion';

test('valid AI response parses correctly', () => {
  const valid = {
    provider: 'mock', confidence: 0.9, looks_like_authorization: true,
    warnings: [], extracted: { paciente: 'Test', dni: '12345678', ... },
    raw_text_preview: '...',
  };
  expect(AutorizacionAIResponseSchema.parse(valid)).toBeDefined();
});

test('invalid provider is rejected', () => {
  const invalid = { ...valid, provider: 'unknown' };
  expect(() => AutorizacionAIResponseSchema.parse(invalid)).toThrow();
});

test('mapAutorizacionToSurgery normalizes DNI', () => {
  const result = mapAutorizacionToSurgery({ ...extracted, dni: '28.456.789' });
  expect(result.dni).toBe('28456789');
});

test('mapAutorizacionToSurgery normalizes DD/MM/YYYY dates', () => {
  const result = mapAutorizacionToSurgery({ ...extracted, fecha_autorizacion: '
15/03/2024' });
  expect(result.fecha_autorizacion).toBe('2024-03-15');
});
```

## 15.3 Prueba visual del formulario

Navegar a /cirugias/nueva. Verificar: (1) la zona de upload se renderiza, (2) subir una imagen de prueba con modo mock muestra el panel de resultados, (3) hacer click en "Aplicar al formulario" precarga todos los campos, (4) las fechas se ven en los campos date, (5) el DNI aparece sin puntos, (6) la tabla de material tiene items, (7) se puede editar cualquier campo, (8) el boton "Guardar" funciona sin errores de consola.

## 15.4 Prueba de errores

- Subir un archivo .txt: debe mostrar error de tipo no soportado.

- Subir un archivo > 20MB: debe mostrar error de tamaño.
- Desconectar internet y subir en modo VLM: debe mostrar error de conexión.
- Subir una imagen que no es una autorización: debe mostrar warning de confianza baja.
- Dejar campos requeridos vacíos y guardar: debe mostrar errores de validación.

## 16. Como documentar en worklog/ADR

La documentación es parte del proceso de desarrollo, no un paso posterior. A continuación se definen los artefactos documentales que deben generarse durante la implementación.

### 16.1 ADR (Architecture Decision Records)

| ADR     | Título                                          | Contenido                                                                                                                                                                                                                                                                       |
|---------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ADR-001 | VLM para extracción de autorizaciones           | Contexto: autorizaciones argentinas semi-estructuradas con 300+ formatos de prestadores. Decision: usar VLM con prompt especializado en lugar de OCR+reglas. Consecuencias: funciona sin configuración por prestador, costo bajo por documento, dependencia de proveedor de IA. |
| ADR-002 | Strategy Pattern para providers de IA           | Contexto: el sandbox uso z-ai-web-dev-sdk acoplado. Decision: interface AIProvider con factory configurable via env var. Consecuencias: cambio de proveedor sin refactor, testing con mock provider, complejidad inicial mayor.                                                 |
| ADR-003 | react-hook-form + Zod para formulario           | Contexto: el sandbox uso useState crudo sin validación. Decision: usar react-hook-form con zodResolver. Consecuencias: validación por campo, dirty tracking, types derivados del schema, curva de aprendizaje.                                                                  |
| ADR-004 | Sin persistencia en DB hasta decisión explícita | Contexto: el sandbox uso store in-memory. Decision: mantener store in-memory inicialmente, no tocar Prisma. Consecuencias: datos se pierden al reiniciar, pero no se contamina el schema de DB.                                                                                 |

Tabla 11: ADRs requeridos

### 16.2 Worklog

Cada paso completado debe registrarse en `/home/z/my-project/worklog.md` con el formato estándar: Task ID, Agent, Task, Work Log (lista de acciones concretas), y Stage Summary (resultados clave, decisiones, artefactos producidos). No avanzar al paso siguiente sin haber registrado el paso actual en el worklog.

### 16.3 README del módulo

Crear `src/lib/services/ai/README.md` con: proposito del modulo, lista de archivos con descripcion, variables de entorno, como ejecutar tests, como agregar un nuevo provider, y link a los ADRs. Este README es la puerta de entrada para cualquier desarrollador que necesite entender o modificar el modulo.

## Checklist ejecutable para Codex/OpenCode

La siguiente checklist esta disenada para ser ejecutada paso a paso por un agente de desarrollo automatizado (Codex, OpenCode, o similar). Cada item es atómico, verificable y no depende de items posteriores. Marcar cada item al completarlo.

- [P1.1] Crear archivo `.env.local` con `AI_PROVIDER=mock`, `AI_TIMEOUT_MS=30000`, `AI_MAX_FILE_SIZE_MB=20`, `AI_CONFIDENCE_THRESHOLD=0.3`
- [P1.2] Verificar que `npm run build` compila sin errores con las nuevas variables
- [P2.1] Crear `src/lib/validators/autorizacion.ts` con schemas Zod y tipos derivados
- [P2.2] Crear `src/lib/validators/cirugia.ts` con `SurgeryFormSchema`
- [P2.3] Crear test unitario que valide parse de JSON de ejemplo y rechazo de JSON invalido
- [P3.1] Crear `src/lib/services/ai/providers/types.ts` con interface `AIProvider`
- [P3.2] Crear `src/lib/services/ai/providers/mock-provider.ts` que implemente `AIProvider`
- [P3.3] Verificar que `mockProvider.extract()` retorna `AutorizacionAIResponse` valido
- [P4.1] Crear `src/lib/services/ai/config.ts` con configuracion centralizada desde env vars
- [P4.2] Crear `src/lib/services/ai/autorizacion-extractor.ts` con factory de providers
- [P4.3] Verificar que `extractAutorizacion()` funciona con `provider=mock`
- [P5.1] Crear `src/app/api/cirugias/ai-extract/route.ts` con POST handler
- [P5.2] Implementar validacion de tipo MIME + extension + tamano
- [P5.3] Implementar timeout con `AbortController`
- [P5.4] Probar con curl en modo mock: verificar respuesta 200 con JSON valido
- [P5.5] Probar con curl archivo invalido: verificar respuesta 400
- [P6.1] Crear `src/lib/services/ai/providers/z-ai-provider.ts`
- [P6.2] Implementar `extract()` con `z-ai-web-dev-sdk createVision()` y prompt mejorado
- [P6.3] Agregar `z-ai` provider al factory
- [P6.4] Probar con imagen real y `AI_PROVIDER=z-ai`
- [P7.1] Crear `src/hooks/useAiExtraction.ts` con estados `idle/loading/success/error`
- [P7.2] Integrar con endpoint `/api/cirugias/ai-extract`

- [P8.1] Crear src/components/cirugias/AiUploadZone.tsx
- [P8.2] Crear src/components/cirugias/AiResultsPanel.tsx
- [P8.3] Crear src/components/cirugias/MaterialTable.tsx
- [P9.1] Crear src/components/cirugias/SurgeryForm.tsx con react-hook-form + zodResolver
- [P9.2] Integrar MaterialTable como campo del formulario
- [P9.3] Implementar recepcion de initialValues para precarga desde IA
- [P10.1] Crear src/app/(dashboard)/cirugias/nueva/page.tsx
- [P10.2] Componer AiUploadZone + AiResultsPanel + SurgeryForm
- [P10.3] Implementar flujo upload, ver resultados, aplicar, editar, guardar
- [P10.4] Usar mapAutorizacionToSurgery() para mapeo IA a formulario
- [P11.1] Crear src/app/api/cirugias/route.ts con POST para guardar cirugia
- [P11.2] Integrar endpoint con boton Guardar del formulario
- [P12.1] Probar flujo completo con modo mock
- [P12.2] Probar flujo completo con modo z-ai
- [P12.3] Probar con archivo invalido, archivo grande, timeout
- [P12.4] Verificar que formulario no se rompe con datos de IA incompletos
- [P13.1] Escribir ADR-001: VLM para extraccion de autorizaciones
- [P13.2] Escribir ADR-002: Strategy Pattern para providers de IA
- [P13.3] Escribir ADR-003: react-hook-form + Zod para formulario
- [P13.4] Escribir ADR-004: Sin persistencia en DB hasta decision explicita
- [P13.5] Crear src/lib/services/ai/README.md
- [P13.6] Registrar cada paso completado en worklog.md